

Linked List

- According to NIST (National Institute of Standards and Technology) linked list is “a list implemented in such a way that each item is having a link to the next item”.
- Linked list and arrays are similar since they both store collections of data.
- The specific type of element is not important since essentially the same structure works to store elements of any type.

Unit 1 - Linked List

Dr. Rabins Porwal

1

- A linked list is an ordered collection of elements. The elements in a linked list are traditionally called nodes. A node in linked list is a structure that has at least two parts: data & link.
 - The data part contains the useful information, i.e., the information to be processed.
 - The link part contains a pointer to the next element in the sequence. It is used to chain the elements together.
 - The link in each element, except the last, points to its successor.
 - The link in the last element contains a null pointer, indicating the end of the list.
 - There is also a pointer variable that identifies the first element in the list.

A linked List with a start pointer

Unit 1 - Linked List

Dr. Rabins Porwal

2

- The simple linked list we have described here is commonly known as a singly linked list because it contains only one link to a single successor.
- The major advantage of the linked list over other general list structures is that data are easily inserted and deleted.
- It is not necessary to shift elements of a linked list to make room to insert a new element or to delete an element as it happens in the case of array.
- On the other hand, because the elements are no longer physically sequenced, we are limited to sequential searches; we cannot use a binary search.

Unit 1 - Linked List

Dr. Rabins Porwal

3

Types of Linked Lists

- Singly-linked List:**

The simplest kind of linked list is a singly-linked list, which has one link per node. This link points to next node in the list, or to null value or empty list if it is the final node.

A singly linked list containing three integer value
- Doubly-linked list:**

A more sophisticated kind of linked list is a doubly-linked list or two-way linked list. Each node has two links: one points to the previous node, or points to a null value or empty list if it is the first node; and the other points to the next, or points to a null value or empty list if it is the final node.

Unit 1 - Linked List

Dr. Rabins Porwal

Unit 1 - Linked List

Dr. Rabins Porwal

4

- Circularly-linked list:**

In a circularly-linked list, the first and final nodes are linked together. This can be done for both singly and doubly linked lists. To traverse a circular linked list, you begin at any node and follow the list in either direction until you return to the original node.

(a) Singly-circularly-linked list:

In a singly-circularly-linked list, each node has one link, similar to an ordinary singly-linked list, except that the next link of the last node points back to the first node. As in a singly-linked list, new nodes can only be efficiently inserted after a node we already have a reference to.

Unit 1 - Linked List

Dr. Rabins Porwal

Unit 1 - Linked List

Dr. Rabins Porwal

5

(b) Doubly-circularly-linked list:

In a doubly-circularly-linked list, each node has two links, similarly to a doubly-linked list, except that the previous link of the first node points to the last node and the next link of the last node points to first node. In a doubly-linked lists, insertions and removals can be done at any point with access to any nearby node.

Unit 1 - Linked List

Dr. Rabins Porwal

Unit 1 - Linked List

Dr. Rabins Porwal

6

4. Header Linked Lists:

A header list is a linked list, which always contain a special node, called header node, at the beginning of the linked list. This header node usually contains vital information about the linked list such as the number of nodes in the list, whether the list is sorted or not.

The diagram shows two types of header linked lists. The first is a 'Header linear linked list, also known as grounded header list', which has a 'Start' pointer to a 'Header node' containing 'X'. This header node points to a sequence of data nodes, each with a pointer to the next node, ending with a null pointer. The second is a 'Circular header list', where the 'Header node' also points to the last node in the sequence, which then points back to the 'Header node', forming a circle.

Unit 1 - Linked List

Dr. Rabins Porwal

7

The diagram shows two more types of header linked lists. The first is a 'Two-way header list', where the 'Header node' points to the first data node, and the last data node points back to the 'Header node'. The second is a 'Two-way circular header list', where the 'Header node' points to the first data node, and the last data node points back to the 'Header node', forming a circle.

Unit 1 - Linked List

Dr. Rabins Porwal

8

Linked List vs. Arrays

Linked lists have several advantages over arrays:

1. Elements can be inserted into linked lists indefinitely, while an array will eventually either filled up or need to be resized, *an expensive operation that may not even be possible if memory is fragmented*. Similarly, an array from which many elements are removed may become wastefully empty or need to be made smaller.
2. Arrays allow random access, while linked lists allow only sequential access to elements.
3. Singly-linked lists, in fact, can only be traversed in one direction.
4. Sequential access in arrays is faster than in linked lists on many machines.

Unit 1 - Linked List

Dr. Rabins Porwal

9

Representation and Implementation of Singly Linked Lists

- **Linked list types:** Before writing the code to build the above list, we need two data types: **Node and pointer**

Node:

- The type for the node which will make up the body of the list.
- Each node contains a single client data element and a pointer to the next node in the list.

Type: struct node

```
typedef struct node{
    int data;
    struct node* next;
}node;
```

- The type for pointers to nodes. This will be the type of the Start pointer and the next fields inside each node. In C and C++, no separate type declaration is required since the pointer type is just the node type followed by a '*'. Type: struct node*

Unit 1 - Linked List

Dr. Rabins Porwal

10

Dynamic Memory Management

Function Name	Description
Malloc	Allocates memory dynamically from heap
Calloc	Allocates memory for specified number of elements
Realloc	Readjust the existing block and copies the contents to the new location
free	Deallocates a block allocated by malloc, calloc & realloc f ⁿ

- **Dynamic Memory Allocation with malloc() function**

The malloc() function dynamically allocates memory. The prototype for malloc() function is

```
ptr = (cast_type *) malloc (byte_size);
```

It takes one argument that specifies the size of the block in bytes. The function returns a pointer to the allocated memory on success or a null pointer(0) in case of failure.

Unit 1 - Linked List

Dr. Rabins Porwal

11

- **Dynamic Memory Allocation with calloc() Function**

The calloc() function dynamically allocates memory for an array of elements. The prototype for calloc() function is

```
void *calloc (n, element_size);
```

It takes two arguments. The first argument is the number of elements and the second argument is size of each element. The function returns a pointer to the allocated memory on success or a null pointer (0) in case of failure.
- **Changing Size of Dynamically Allocated Memory with realloc() function**

The realloc() function changes the size of previously dynamically allocated memory with malloc(), calloc() or realloc() functions. The original contents held in the previously allocated memory are not changed provided the memory allocated is larger than the amount of memory previously allocated otherwise the contents are unchanged upto the size of new object. The prototype for realloc() is fⁿ

```
void *realloc( void *block, size_t size);
```

It takes two arguments. The first argument is the pointer to the original object and the second argument is new size of the object.

Unit 1 - Linked List

Dr. Rabins Porwal

12

• Deallocating Memory with free() function

The free() function deallocates a memory block previously allocated with malloc(), calloc() or realloc() functions. The prototype for free() function is

```
void free(void *block);
```

It takes one argument that specifies the pointer to the allocated block. But it is important to note that only the allocated block is deallocated, the pointer variable is not deleted.

Once the block is freed, an attempt to dereference a pointer variable yield undefined result. Such a pointer variable is called a **dangling pointer**. Therefore, it is recommended that after freeing the allocated block, set the pointer variable to NULL explicitly.

Unit 1 - Linked List

Dr. Rabins Porwal

13

Algorithm for traversing a Linked List

Let LIST be a linked list in memory. This algorithm traverse LIST, applying an operation PROCESS to each element of LIST. The variable PTR points to the node currently being processed.

1. Set PTR:=START
2. Repeat steps 3 and 4 while PTR!=NULL
3. Apply PROCESS to INFO[PTR]
4. Set PTR:=LINK[PTR] (*PTR now points to next node*)
[End of step 2 loop]
5. Exit.

Unit 1 - Linked List

Dr. Rabins Porwal

14

Algorithm for searching a Linked List

Search (INFO, LINK, START, ITEM, LOC)

This algorithm finds location LOC of the node where ITEM first appears in LIST, or sets LOC = NULL

1. Set PTR:=START
2. Repeat step 3 while PTR!=NULL
3. If ITEM=INFO[PTR], then
Set LOC:=PTR, and exit
Else:
Set PTR:=LINK[PTR]
[End of if structure]
4. [Search is unsuccessful] Set LOC:=NULL
5. Exit

Unit 1 - Linked List

Dr. Rabins Porwal

15

Memory Allocation

The maintenance of linked list in memory assumes possibility of inserting new nodes into the lists and so it requires some mechanism which provides unused memory space for the new node. Also some mechanism is required so that the memory space of deleted nodes becomes available for future use.

- So along with the linked list in memory, a special list is maintained which consists of unused memory cells.
- This list which has its own pointer, is called the **list of available space** or the **free storage list** or the **free pool**.

Unit 1 - Linked List

Dr. Rabins Porwal

16

Garbage Collection

If a node is deleted from the list or the entire list is deleted from the main program. Then we want that the memory space should become reusable. To make the space available for future use, there are two ways:

1. One way is to immediately insert the list into the free storage list. This method is implemented while using linked list by means of linear arrays. But this is a time consuming process for the operating system of a computer.
2. The other method is that the OS may periodically collect all the deleted space onto the free storage list. The technique which does this collection is called the **Garbage Collection**.

Unit 1 - Linked List

Dr. Rabins Porwal

17

Garbage Collection usually takes place in two steps:

1. First the computer runs through all the lists, tagging those cells which are currently in use.
 2. Then the computer runs through the memory, collecting all untagged spaces onto the free storage list.
- Garbage Collection may take place when there is only some minimum amount of space or no space at all left in the free storage list, or when the CPU is idle and has time to do the collection.
 - It is invisible to the programmer.

Unit 1 - Linked List

Dr. Rabins Porwal

18